Week 1: Project Proposal

Anh Huynh

Project Pitch

Game Title: Epic Brawlers

Description:
Epic Brawlers is a click-to-fight boss game built with Java Swing. The player faces one enemy boss at a time. Clicking **attack** inflicts damage; defeating a boss awards gold, which the player can spend on **weapon upgrades** to increase damage per click. Selecting to move onto the **Next Boss** spawns a tougher opponent with higher health and larger rewards.

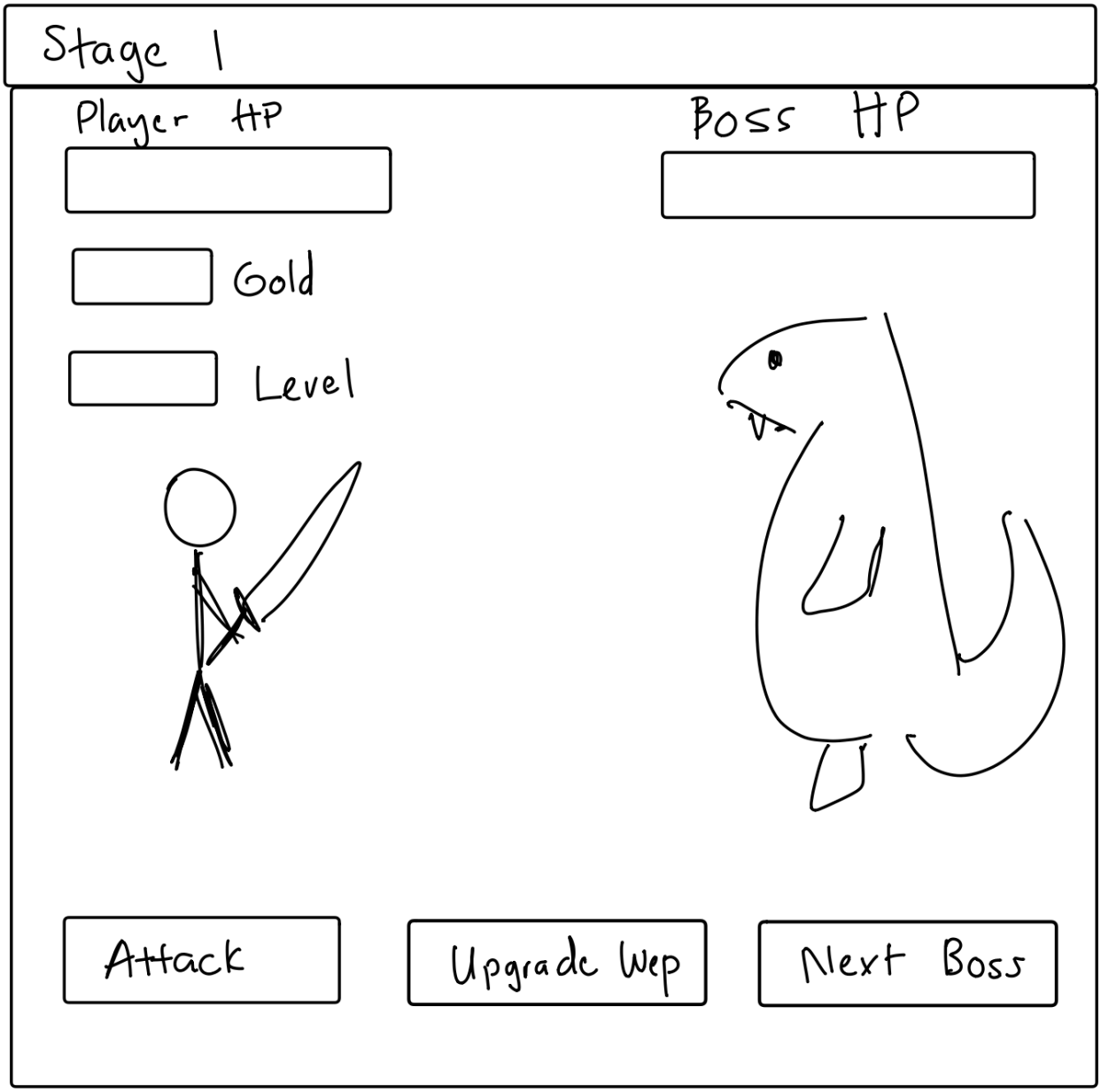
All gameplay happens through GUI components — buttons, labels, and a progress bar. There's no player movement; instead, progression focuses on stages, upgrades, and incremental strength.

Gameplay each round:
When the player clicks Attack, the player deals damage to the boss, and immediately after, the boss attacks back. This ensures that both sides lose HP each round. When the boss's HP reaches 0, the player earns gold and has the option to upgrade their weapon (upgrades unavailable if funds are insufficient) or advance to the next stage, where the next boss has higher HP, greater attack damage, and better rewards.

User actions:

- Clicks **Attack** —> Boss HP decreases. If HP <= 0, the boss dies, the player earns gold, and a new boss appears.
- Clicks **Upgrade Weapon** —> if enough gold, the player's weapon level increases, raising it's attack damage
- Clicks **Next Boss** —> Generates the next-tier boss.
- **Stage Progression** —> defeating a boss allows to advance to the next stage
- **Death/Reset** —> if HP <= 0, the player restarts at Stage 1, fully healed.

GUI Preview



CRC Cards

Player: tracks gold, level, damage, hp; handles upgrades and taking damage.
Collaborator(s): *Boss*

Collaborator(s): *Boss*

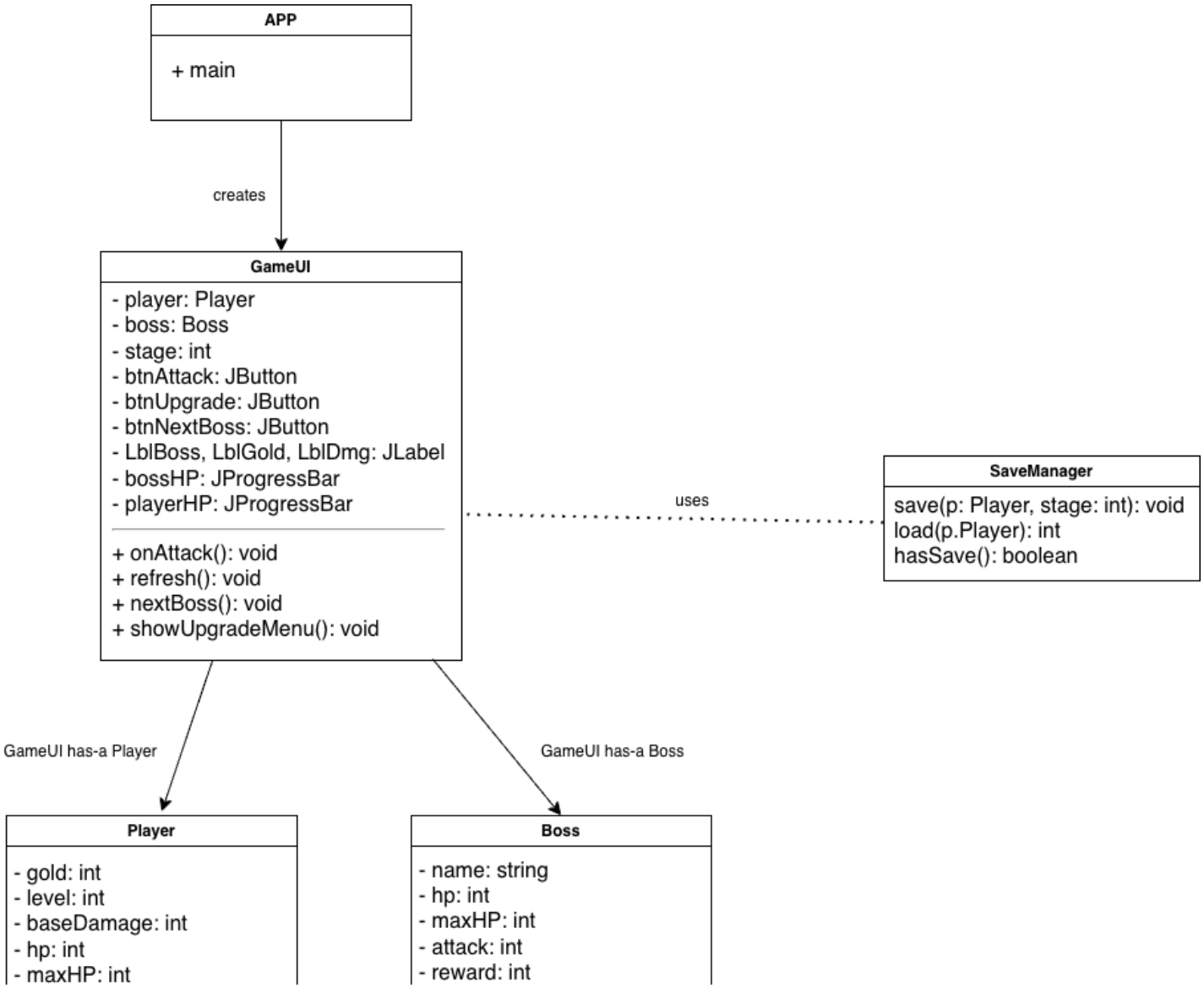
Boss: stores hp, maxHp, reward, attack, stage; handles taking damage
Collaborator(s): *Player*

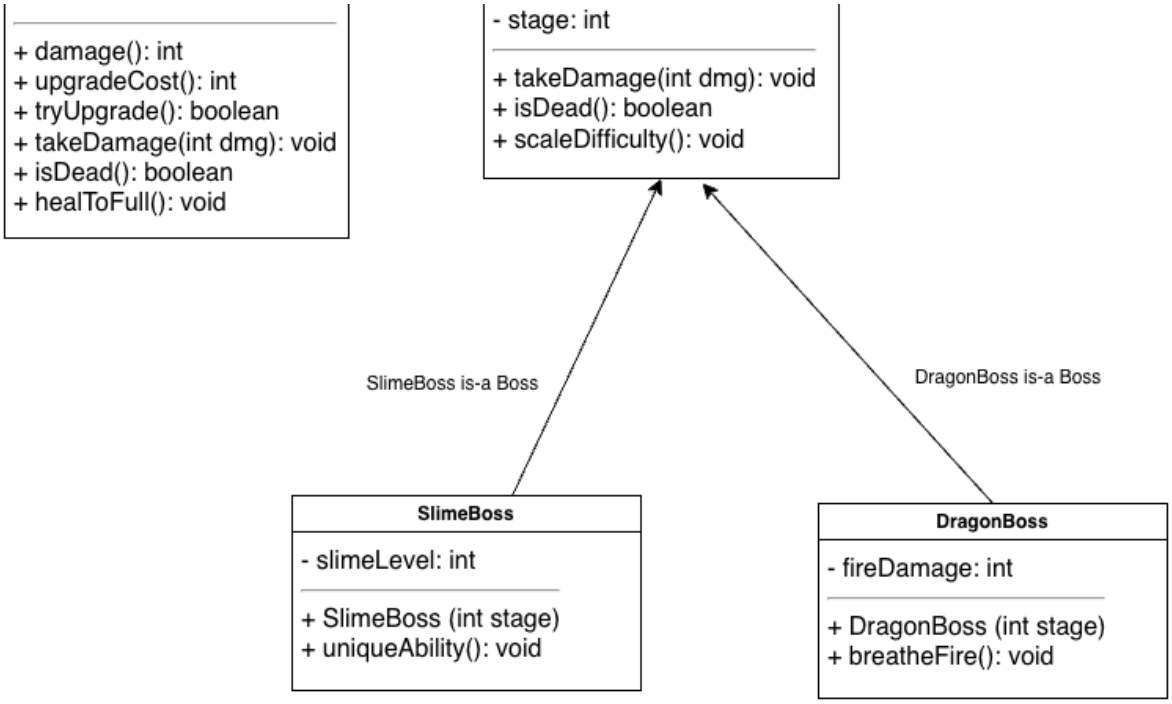
GameUI: manages GUI display and logic; responds to button clicks; updates HP bars and stage progression
Collaborator(s): *Player, Boss*

***BaseWeapon:** base class for all weapons; defines common fields (name, damage, level) and methods for attacking and upgrading

Collaborator(s): *Player, Boss*
*= possibly added in future

UML Diagram:





Object-Oriented Design

- Aggregation: The GameUI has-a Player and a Boss
- Encapsulation: Each class manages its own state and behavior (damage, HP, gold).
- Polymorphism: future versions include subclasses such as a SlimeBoss and DragonBoss to demonstrate inheritance
- Event-Driven Programming: each click triggers ActionListener events, updating the GUI and progressing the game round-by-round

Video Link: https://youtu.be/fAck9a8_tyI

Learning Outcomes

LO	Demonstrated in Epic Brawlers
LO1: Object Oriented Design Principles	Clear separation of classes; encapsulated Player/Boss classes; GUI/controller in GameUI class
LO2: · LO2: Construct programs utilizing single and multidimensional arrays	Optional

(optional)	
LO3: Objects & Aggregation	GameUI aggregates Player and Boss and coordinates their interactions
LO4: Inheritance & polymorphism	Planned boss hierarchy (e.g., BaseBoss —> SlimeBoss, DragonBoss) without changing controller code
LO6: GUI & Event-Driven Programming	Swing (JFrame, JButton, JLabel, JProgressBar) with ActionListener callbacks for attacks/upgrades
LO7: Exception handling	Handling for insufficient funds on upgrade; safe HP bounds; death/reset flow
LO8: Text File I/O	Save/load player progress (gold, level) to a simple text file

Planned Working Time

Week	**Hours (outside class)**
Week 1	Thurs 2-5 PM + Fri 6-9 PM
Week 2	Thurs 2-5 PM + Fri 6-9 PM
Week 3	Thurs 2-5 PM + Fri 6-9 PM / Sat 11 AM - 2 PM
Week 4	Thurs 2-5 PM + Fri 6-9 PM / Sat 11 AM - 2 PM
"..."	"..."

TO-DO Timeline Plan

Week	**Goals**
1	Write proposal; design CRC cards and UML; set up project files.
2	Implement Player and Boss classes; test damage logic and gold system.

3	Finalize model logic; create beta GUI layout. Implement rough codes for all other classes
4	Test and refine attack flow; prepare a simple non-functional GUI.
5	Test the GUI (view) and display live HP updates.
6	Connect event handling (controller) for attack and upgrade buttons.
7	Debug and polish stage progression and death/reset behavior.
8	Finalize all documentation and code; thorough testing

GitHub Repository

https://github.com/anhpls/epicbrawlers

Week 1 Additional Deliverables:

· *Deliverable 2 (optional, on the Canvas Project submission):* If you have started writing code, submit the code you have written so far, even if it is not complete and/or has compiler errors.

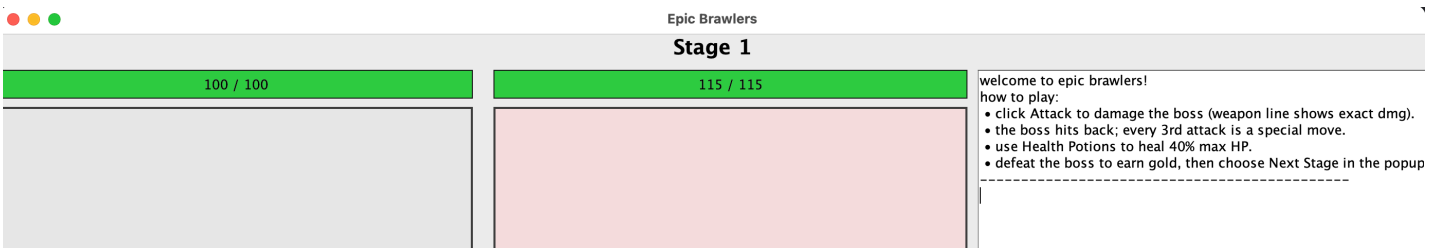
Week 2: Updates

Revise anything that is needed in the Week 1 section based on instructor feedback. Make sure your design is viable before working on your Week 2 goals from your project timeline.

Week 2 Deliverables

Deliverable 1 (on the Canvas Project submission): Submit the code you have written so far, even if it is not complete and/or has compiler errors.

Deliverable 2 (here): Update your Canvas Project page from Week 1. Please add to the page - do not delete any content from Week 1:



Journal Entry

This week, I focused on implementing the weapon classes and refining how damage scaling works across different weapon levels. Each weapon (Stick, Bow, Sword, and Magic Staff) now scales its damage using an exponential growth formula, which makes upgrades feel more impactful as the player progresses. I tweaked my original game design to make character progression more interesting. Instead of a straightforward “defeat boss → move to next stage” flow, I added the idea of upgrading weapons and revisiting older bosses** to grind for gold and build up strength through upgrades. This change adds more strategy to the gameplay and makes progression feel more rewarding.

Design Changes

The biggest design change this week was shifting the focus from a linear boss-fight system to a more upgrade-based loop. Players can now choose to replay stages to earn gold, purchase weapon upgrades, and boost their survivability before moving on to tougher bosses. This aligns better with idle and incremental RPG mechanics and gives players more freedom to decide how to progress.

Additionally, I improved the internal class design by introducing BaseWeapon and BaseBoss as parent classes. This structure uses inheritance and polymorphism to make it easier to define unique behavior for each subclass while sharing core logic like health, attack, and scaling.

Challenges Encountered

Balancing the damage output between the player and bosses was one of the main challenges. Early versions of the scaling system made weapons either too weak or too overpowered at certain levels. I also had to carefully adjust each boss’s special attack logic to make sure battles stayed fair but challenging. (Still a work in progress). Another issue was managing the interaction between the player’s stats and boss behavior in a way that kept the gameplay loop consistent.

What's Next

Next week, I plan to:

- Improve the **GUI** for the battle screen.

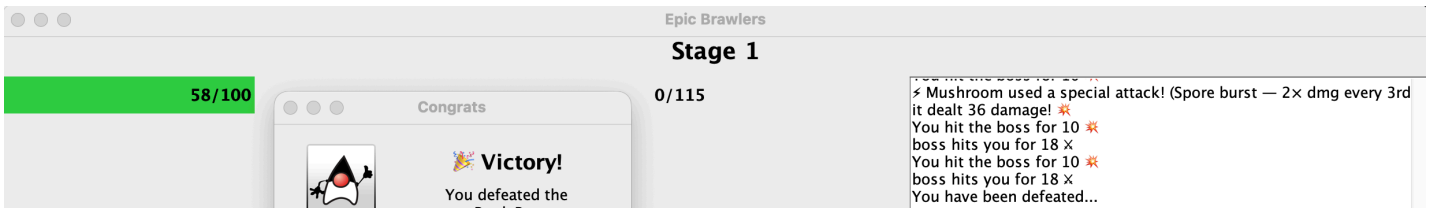
- Display **health bars, gold, potions, and weapons** visually for better clarity.
- Add early versions of the **shop system** so players can spend gold on upgrades.
- Begin connecting player and boss logic through an early controller class.

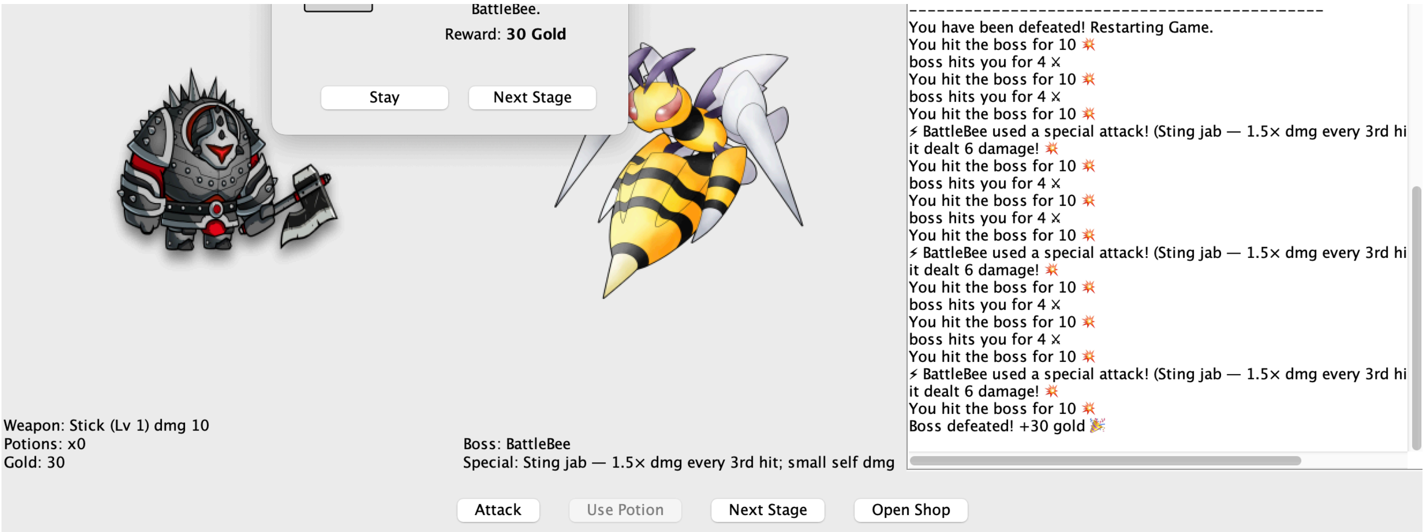
Updated Timeline

Week	Goals
1	Write proposal; design CRC cards and UML; set up project files.
2	Implement Player and Boss classes; test damage logic and gold system.
3	Finalize model logic; improve GUI layout; possibly add intro screens to give backstory to the game; implement rough codes for all other functionalities (mainly the shop + save/load progress)
4	(heavy focus on backend) Test and refine attack flow; heavy testing on gameplay logic + design and character progression
5	(focus on frontend) test the GUI for bugs + improve GUI interactivity + include better visuals
6	Connect event handling (controller) for attack and upgrade buttons.
7	lots of time debugging + improving gameplay logic before final touches.
8	Finalize all documentation and code; thorough testing

Week 3: Updates

Week 3 Deliverables





Journal Entry

This week, I focused heavily on refining the Beta UI and fixing a long list of small but important issues that appeared once the battle screen became more visually complex. I spent time adjusting spacing, font sizes, and panel layouts to make the interface cleaner and easier to understand. I also added player and boss images, which required fixing scaling issues so the sprites would display correctly without stretching or overflowing their panels

Several interactive components needed logic fixes as well. For example, the “Use Potion” button now correctly disables itself when the player has no potions, and the “Next Stage” button only becomes enabled once the current boss is defeated. I also resolved problems with the health bars, which were initially not showing 0 HP correctly, or were displaying incorrect values after taking damage.

Week 3 was less about adding new gameplay mechanics and more about improving visual clarity and refining how the UI responds to game state changes, which helps the game feel more polished and functional.

Design Changes Made

- I replaced text labels for the player and bosses with actual images, making the game screen look more like a real RPG.
- I removed background panels and borders around the icons to create a cleaner, minimalist battle layout.
- I restructured parts of the UI logic so that boss images, stage labels, and button states update immediately whenever the player progresses or restarts.
- I added early support for a Re-Battle option where players can challenge previously defeated bosses to grind gold without advancing the stage.

These changes were made to make the game feel visually clearer and more interactive, and to better support the gameplay loop centered around progression and upgrading.

Challenges Encountered

This week's biggest challenges were UI-related:

- Image scaling caused stretched or oversized boss icons until I implemented a scaling helper method.
- The health bars didn't update properly at first; some values didn't display 0 HP or overflowed incorrectly.
- Button logic needed careful handling so players couldn't press actions at the wrong time (e.g., using potions when none were available).
- Some layout panels overlapped or misaligned, so I spent time adjusting layout managers, padding, and component sizes.

Debugging UI issues took more time than expected, but it helped solidify the underlying structure of the game.

What I Still Need to Do

To complete the project, I still need to:

- Implement the Shop System so players can spend gold on weapon upgrades and stats.
- Add Save/Load features or at least persistent player stats.
- Build more polished intro screens or story panels.
- Add more visual polish such as animations, effects, or sound (if time allows).
- Begin balancing boss stats, rewards, and difficulty curves.

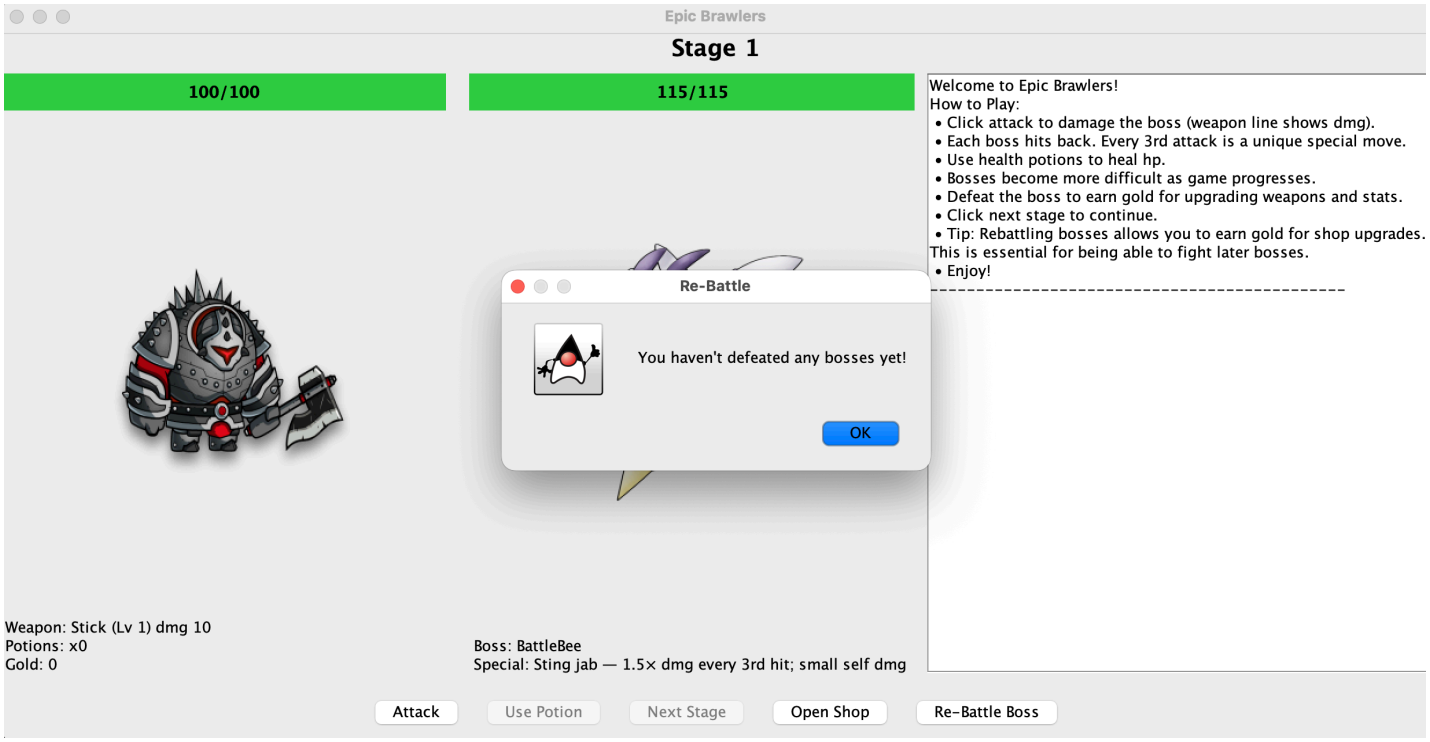
Updated Timeline

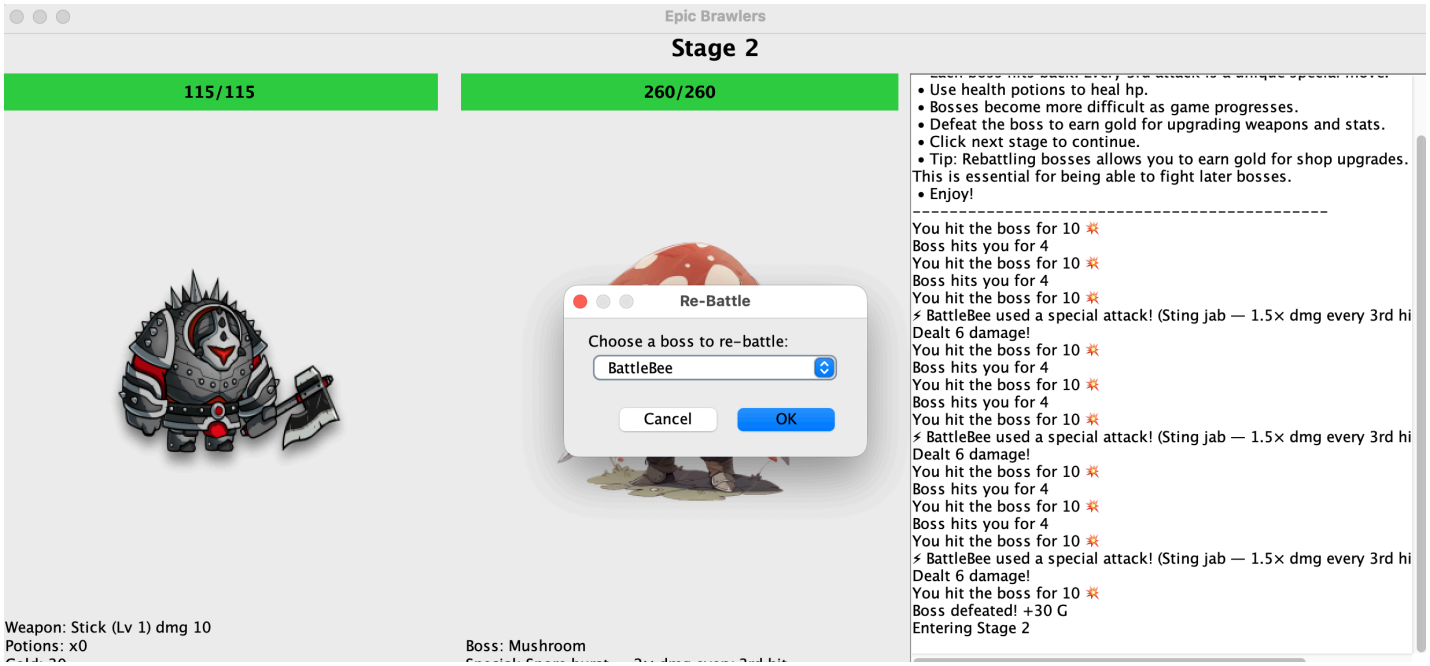
Week	Goals
1	Write proposal; design CRC cards and UML; set up project files.
2	Implement Player and Boss classes; test damage logic and gold system.
3	Finalize model logic; improve GUI layout; possibly add intro screens to give backstory to the game; implement rough codes for all other functionalities
4	(heavy focus on GUI) Test and refine attack flow; heavy testing on GUI + include characters as well as enemies
5	(focus on frontend) test the GUI for bugs + improve GUI interactivity + include better visuals
6	Connect event handling (controller) for attack and upgrade buttons.

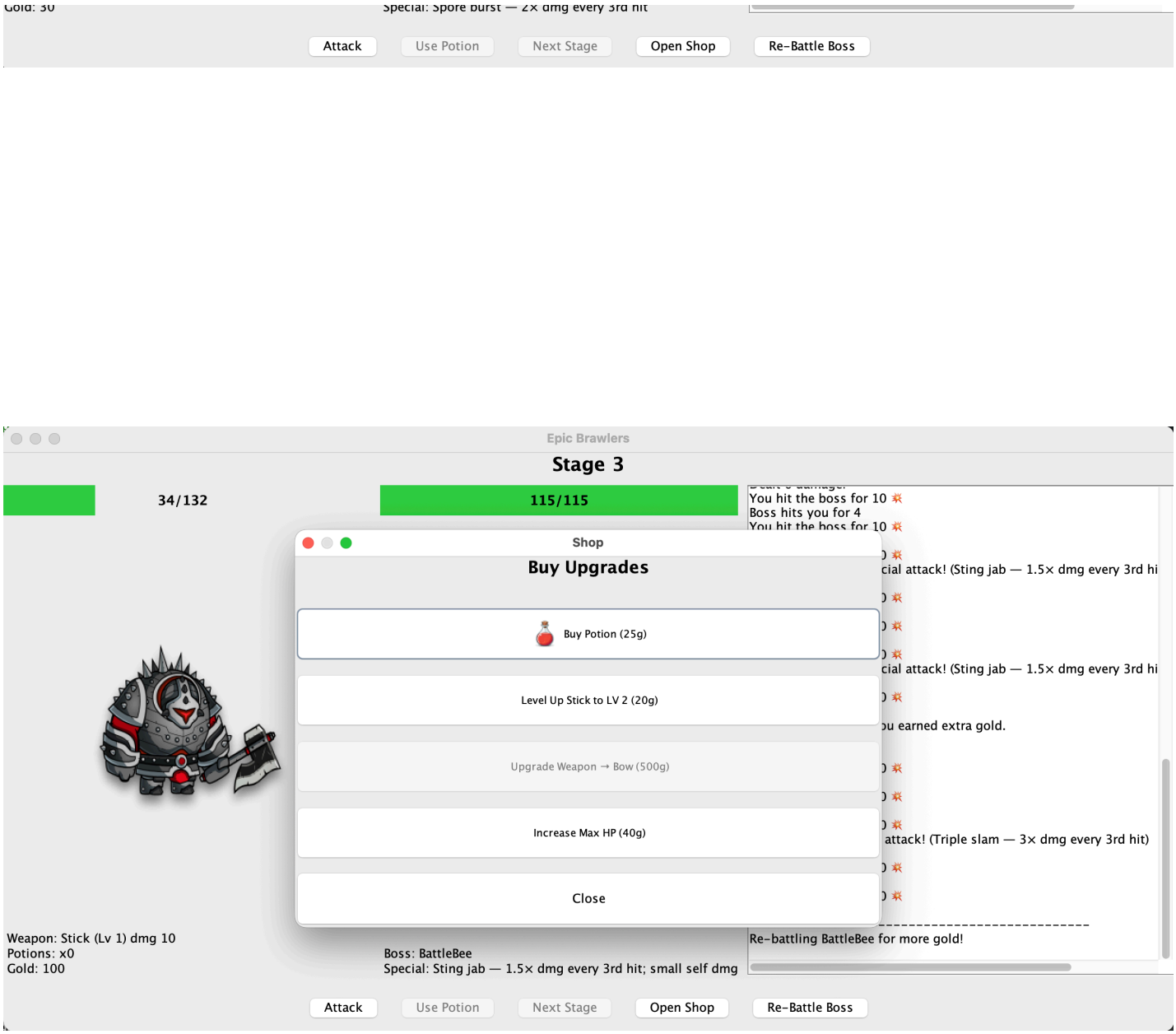
7	lots of time debugging + improving gameplay logic before final touches.
8	Finalize all documentation and code; thorough testing

Week 4: Updates

Week 4 Deliverables







Tested weapon upgrades logic + cycling through bosses + gold farming + highest stage reached.

Journal Entry

This week, I shifted my focus from the battle system to building a fully functional Shop system, which is now a central part of the game's progression loop. The Shop allows players to spend their earned gold to upgrade weapons, increase their max HP, and buy health potions. This required designing both the UI for the shop window and the internal logic for each upgrade path, especially since weapons now have two separate upgrade tracks: leveling up the current weapon and advancing to the next weapon tier (Stick → Bow → Sword → Magic Staff).

A large part of the work involved making the shop feel responsive to the player's current stats. Every purchase instantly updates the main UI so the player sees their new gold, weapon levels, or HP right away. Getting the buttons to enable/disable correctly depending on how much gold the player has also took careful coordination between shop.java, player.java, and the main.ui

took careful coordination between shop.java, player.java, and the main UI.

Week 4 centered around adding progression mechanics and making the shop functional.

Design Changes Made

- I introduced two distinct weapon upgrade systems:
    - **Level Up Weapon** → increases the level of the currently equipped weapon (infinite scaling).
    - **Upgrade Weapon Tier** → switches the player to a stronger weapon class entirely.
- I added dynamic button text in the shop, which updates based on:
    - the player's gold,
    - current weapon level,
    - whether the next tier exists,
    - and which upgrades are affordable.

- Shop actions now immediately refresh the main UI, ensuring the player can see updated gold, HP, and damage values without closing the shop.
- I added icons to the shop buttons (potions, weapons, HP, close button) to make the window more visually appealing and readable.
- Weapon tier upgrades were designed to be significantly more expensive, encouraging players to grind and naturally progress through battles.
- Player stats now scale smoothly by allowing max HP upgrades through the shop.

These changes improve player choice and make the gameplay loop more strategic; players must decide whether to invest in more HP, improve their weapon, upgrade weapon tiers, or stack potions before fighting harder bosses.

Challenges Encountered

This week's challenges were more logic-oriented than visual:

- Ensuring the shop buttons always show the correct text and prices required adding a full refresh system (refreshShop()), since weapon tiers and levels constantly change.
- Balancing weapon tier costs so the player cannot switch too early.
- Making the shop window modal (forcing the player to finish interacting with it before returning to the game) initially caused UI update delays until I synced refreshing between the shop and the main UI.
- Handling two upgrade paths (level ups vs. tier changes) complicated the button logic and required new helper methods to determine the next weapon tier and its price.

- Managing icons and paths required experimenting with image scaling so that graphics didn't blur or distort.

What I Still Need to Do

- Add tier-based scaling for enemies to match the stronger weapons.
- Continue adjusting prices and scaling formulas so the difficulty remains balanced.
- Implement a save/load system so upgrades persist across sessions.

Updated Timeline

Week	Goals
1	Write proposal; design CRC cards and UML; set up project files.
2	Implement Player and Boss classes; test damage logic and gold system.
3	Finalize model logic; improve GUI layout; possibly add intro screens to give backstory to the game; implement rough codes for all other functionalities
4	(heavy focus on GUI) Test and refine attack flow; heavy testing on GUI + include characters as well as enemies; include re-battle logic for gold accumulation; fix any logical issues with rebattling bosses
5	(focus on frontend) test the GUI for bugs + improve GUI interactivity + include better visuals; fix up shop and functionalities of shop; include option to change weapons for more damage; look at looping bosses logic
6	Connect event handling (controller) for attack and upgrade buttons; **TBA** - will see how progress is and what else to work on
7	lots of time debugging + improving gameplay logic before final touches.

8	Finalize all documentation and code; thorough testing

Week 5: Updates

Week 5 Deliverables

Discovered an error of damage decreasing after leveling up weapon a bunch of times.

```
Entering Stage 46
You hit the boss for 842891838 💥
Boss defeated! +3,245 G
...
Entering Stage 49
You hit the boss for 52734784 💥
Boss defeated! +3,455 G
```

Full Error Log:

```
You hit the boss for 842891838 💥
Boss defeated! +3,035 G
Entering Stage 44
You hit the boss for 842891838 💥
Boss defeated! +1,790 G
Entering Stage 45
You hit the boss for 842891838 💥
Boss defeated! +9,000 G
Entering Stage 46
You hit the boss for 842891838 💥   <-------- before a lot of level ups
Boss defeated! +3,245 G
Entering Stage 47
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
```

```
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Your Magic Staff leveled up!
Max HP increased!
Max HP increased!
Max HP increased!
Max HP increased!
Max HP increased!
Max HP increased!
Max HP increased!
Max HP increased!
Max HP increased!
Max HP increased!
Max HP increased!
Max HP increased!
Max HP increased!
Max HP increased!

Max HP increased!
Max HP increased!
Max HP increased!
Max HP increased!
Max HP increased!
Max HP increased!
Max HP increased!
Max HP increased!
Max HP increased!
Max HP increased!
```

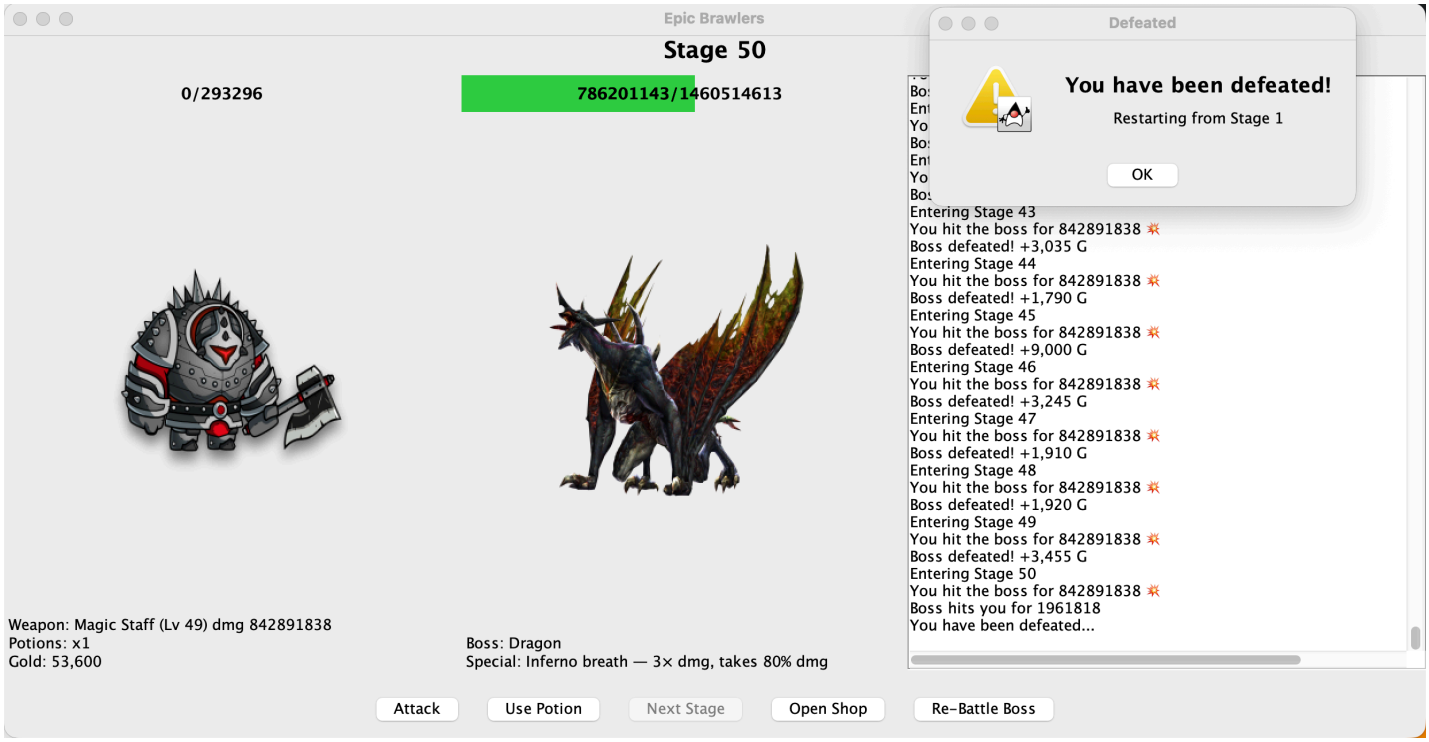
```
Max HP increased!
Max HP increased!
Max HP increased!
Max HP increased!
Max HP increased!
Max HP increased!
Max HP increased!
Max HP increased!
You hit the boss for 52734784 💥
Boss defeated! +1,910 G
Entering Stage 48
You hit the boss for 52734784 💥
Boss defeated! +1,920 G
Entering Stage 49
You hit the boss for 52734784 💥    <------ lower than before (842891838)
Boss defeated! +3,455 G
Entering Stage 50
```

Error Log Notices:

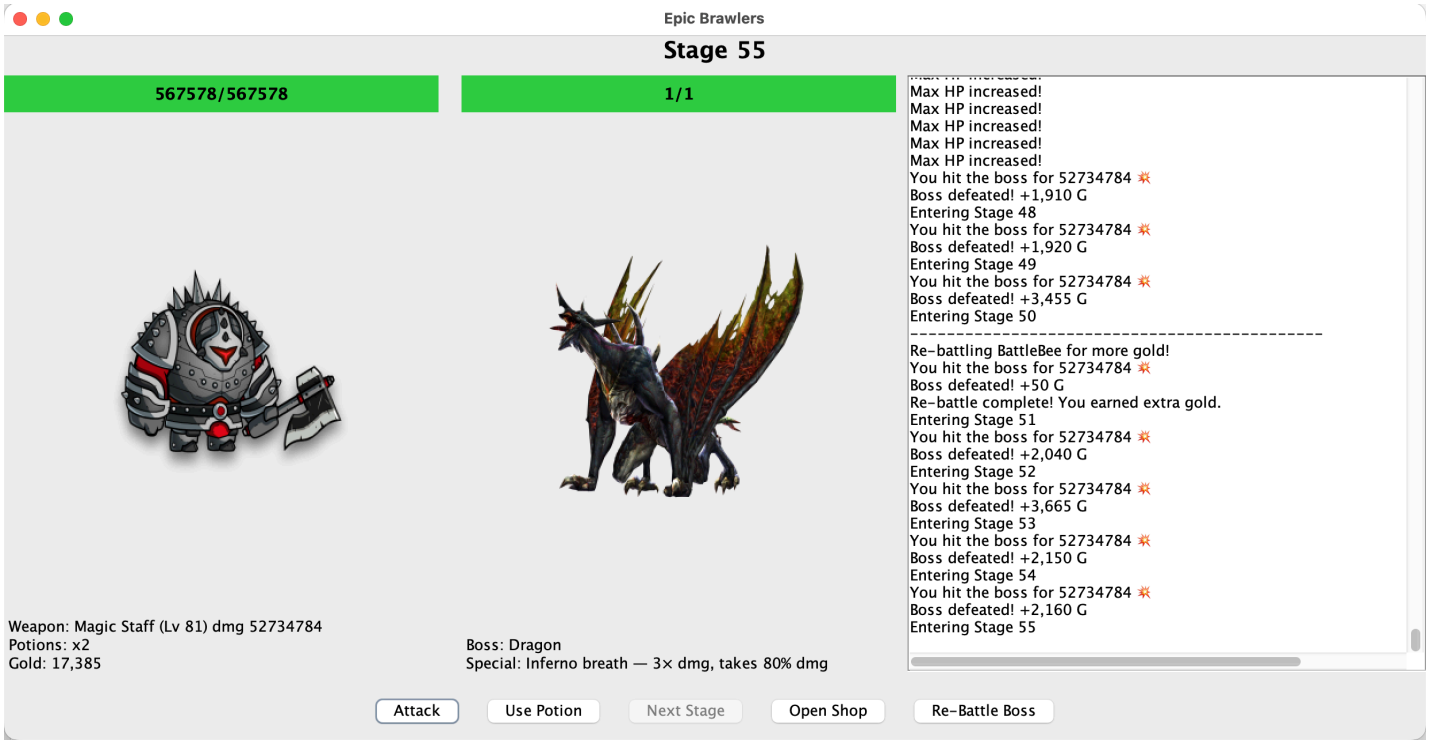
- Later stages have more errors for some reason
- Dragon boss with 1 hp around stage 50+
- Other bosses do not scale properly with stage progress; only boss that does is Slime Boss
- Dmg descriptor next to weapon name is not showing appropriate dmg; maybe because of spacing within the UI?
    - a fix could be displaying commas for the dmg instead of a row of just numbers
        - showing dmg is less?? after leveling up a bunch of times than before?? but not too sure
- maxHP upgrades in shop become very minimal in later stages; maybe needs to scale somehow
    - maybe as stages progress, percentage of maxHP obtained scales more because what it's currently at is not enough.
- sometimes, next stage button gets glitched and only way to get out is re-battling
- Damage is stuck at 1? A lot of random values dropping to 1 after stage 50
    - Bosses do not give rewards during this error, even when rebattling
    - Stuck at 1 dmg, 0 gold, 0 health pots — this is where the game technically "crashes"

      because no progression could be made
        - possible fix (save game, exit, and reload progress?)

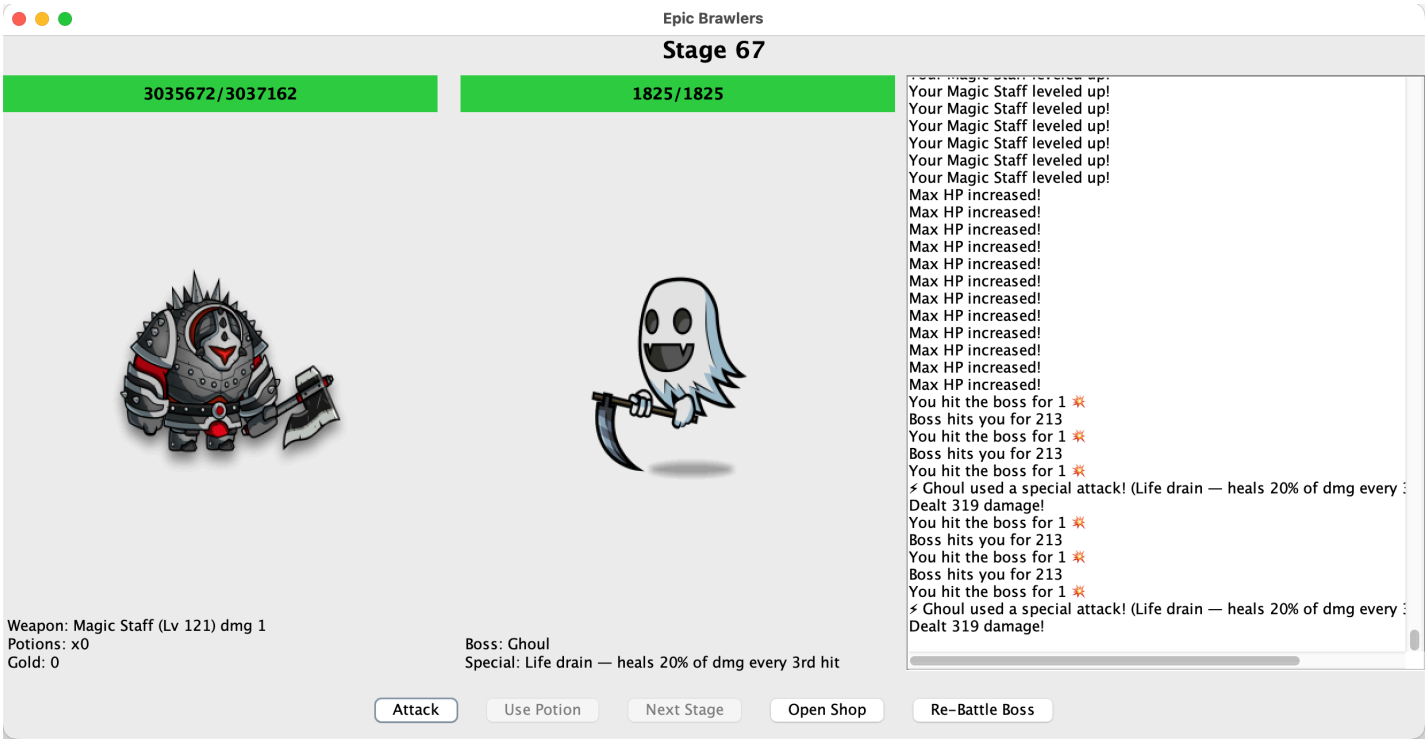
Example of Late Game Progression:



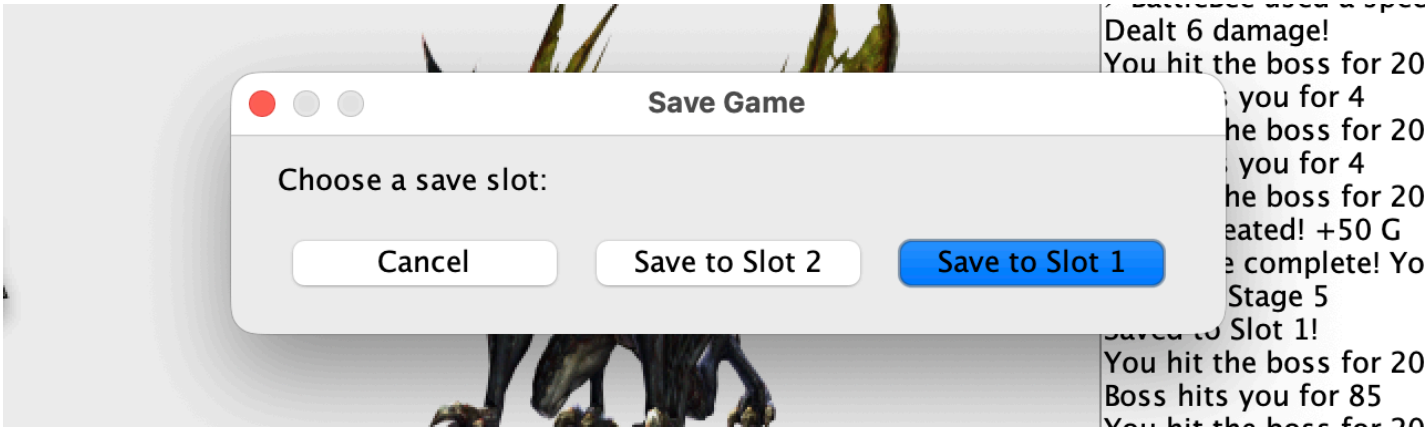
Dragon Boss with 1 HP after Stages 50+ Error Log:

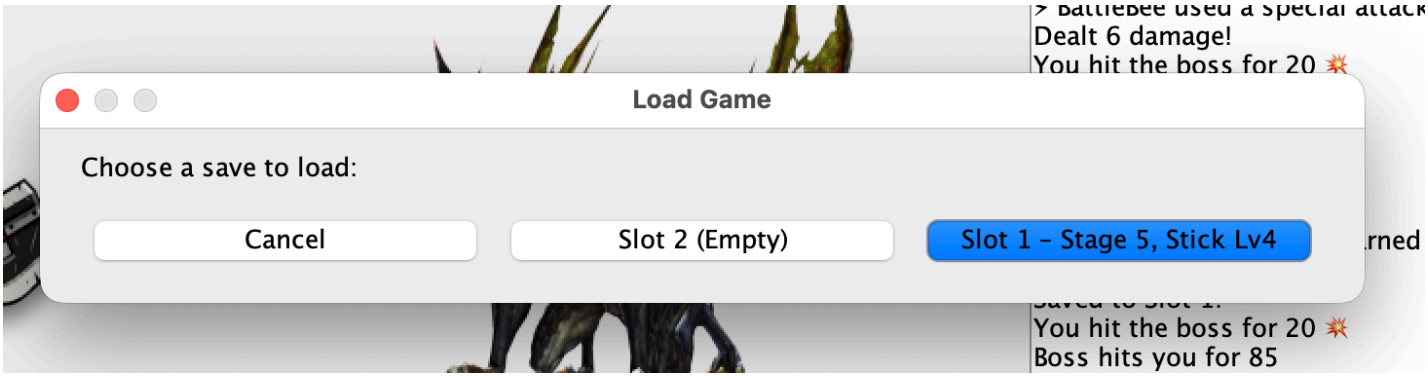


1 Damage Error + 0 rewards for defeating Bosses at this point:



Save / Load Game I/O + UI:





Journal Entry

This week, I focused on adding a complete Save / Load System using Java I/O, while also debugging some major late-game scaling issues that appeared once weapons reached extremely high levels. I made several UI improvements and implemented two separate save slots, allowing the player to store different progressions and load them when launching the game.

At the same time, I stress-tested the game up to Stage 50+, which revealed a number of issues with integer overflow, boss scaling, and inconsistent UI updating. These issues helped me identify areas that need to be changed going into next week.

Design Changes Made

1. **Added Save / Load Game System using Java Serialization (for the I/O requirement)**
    - implemented a SaveData class
    - The game now writes player stats, current stage, weapon type/level, maxHP, and current HP into save1.dat or save2.dat.
    - When loading, the game restores the entire state:
        - stage
        - HP / maxHP
        - weapon type + level
        - potion count
        - gold
        - correct boss for that stage
2. **Added Multi-Slot Save Menu (better UI + adds progression support)**
    - Save Menu now shows (need to fix the order of options):
        - Save to Slot 1
        - Save to Slot 2
        - Cancel
    - Load Menu shows descriptive labels so the user knows what's inside each slot:
        - Slot 1 — Stage 31, Magic Staff Lv14
        - Slot 2 — Empty
3. **Identified and Documented Critical Late-Game Errors**

    During testing beyond Stage 40–50, several unexpected behaviors appeared:

Damage Overflow / Damage Resetting to Lower Values

- Exponential growth in getDamage() using Math.pow() causes:
- numbers too large to fit in int
- overflow wraps values back down into smaller numbers (sometimes even negative internally before clamping)
- damage displayed becomes incorrect, especially after many upgrades

Damage Dropping to “1” Permanently

- Once overflow/underflow happens, the calculation becomes unstable
- after Level 40–50+ on Magic Staff, damage returned from Math.pow() starts returning Infinity / NaN internally
- the Math.max(1, ...) clamps dmg to 1
- This causes:
- player dealing only 1 dmg
- bosses giving 0 gold
- potion count stays 0
- game progression becomes impossible
- effectively a “soft crash”

Boss HP Overflow

- Dragon appears with 1 HP at later stages
- This happens because boss scaling is also using exponential/int values which overflow to extremely small (or negative) ranges

UI Weapon Damage Label Does Not Update Properly

- Long numbers overflow the JLabel space
- The UI may truncate or hide digits

Challenges Encountered

1. Integer Overflow in Damage Scaling

Once weapon level gets too high (30–40+), exponential growth overflows Java’s `int` range.

Symptoms:

- Damage suddenly becomes smaller
- Eventually collapses to 1
- Boss HP collapses
- Boss rewards drop to 0
- Game progression breaks completely

This was the largest technical issue this week.

2. MaxHP Upgrade Scaling Too Weak

- Early scaling felt fine (lots of rebattling needed to progress)
- Mid-game (Stage 20–35): upgrades are very helpful and player still needs to carefully think about stats vs wep to upgrade
- Late-game: maxHP increase is too small relative to maxHP upgrade buy button because constant 40g even though gold gets into the thousands
- Player becomes paper-thin at high stages without proper maxHP upgrades (might need to add defense upgrade)

3. “Next Stage” Button Glitches

- Sometimes disables itself incorrectly
- Only fix is rebattling an earlier boss

Might be an issue between:

- `rebattleMode` flag
- defeat popup logic
- `enableNextStage()` triggers

4. UI Overflow Issues

- Long damage numbers (e.g., `842,891,838`) break JLabel formatting
- Hard to read; sometimes doesn’t show at all
- Comma-formatting or abbreviating numbers (e.g., `842M`) would help

---

What I Still Need To Do

1. Fix Weapon Damage Overflow (Critical + Priority)

- Switch from `int` to `long` or `BigInteger`
- Or cap damage scaling before overflow happens
- Or move to logarithmic or soft-capped formulas
- Must be fixed before progression can continue past Stage 40

2. Fix Boss HP Scaling

- Prevent HP overflow in late stages
- Ensure dragons and other bosses scale consistently
- Make HP increases match late-game damage

3. Fix Next Stage Button Bug

- Add explicit state reset logic
- Ensure `rebattleMode` toggles off correctly
- Ensure defeat popups don't interfere with UI state
- Guarantee `enableNextStage()` is always triggered after boss defeat

4. Improve UI Stats Formatting

- Add comma formatting to large numbers
- Or abbreviate big values (ex: `1.2M`, `530K`, etc.)
- Increase label width or adjust layout spacing

5. Rebalance MaxHP Upgrades

- Increase scaling with stage progression
- Possibly scale % increase based on:
    - stage number
    - weapon level
    - shop tier
- Ensure HP keeps up with boss scaling in late game

6. Add Additional Save Slot Safety

- Auto-save after every boss defeat
- Ask for confirmation before overwriting slots
- Display last played timestamp for each save slot
- Reorder the button options in popup menus (Save/Load Menus)

Updated Timeline

Week	Goals
1	Write proposal; design CRC cards and UML; set up project files.

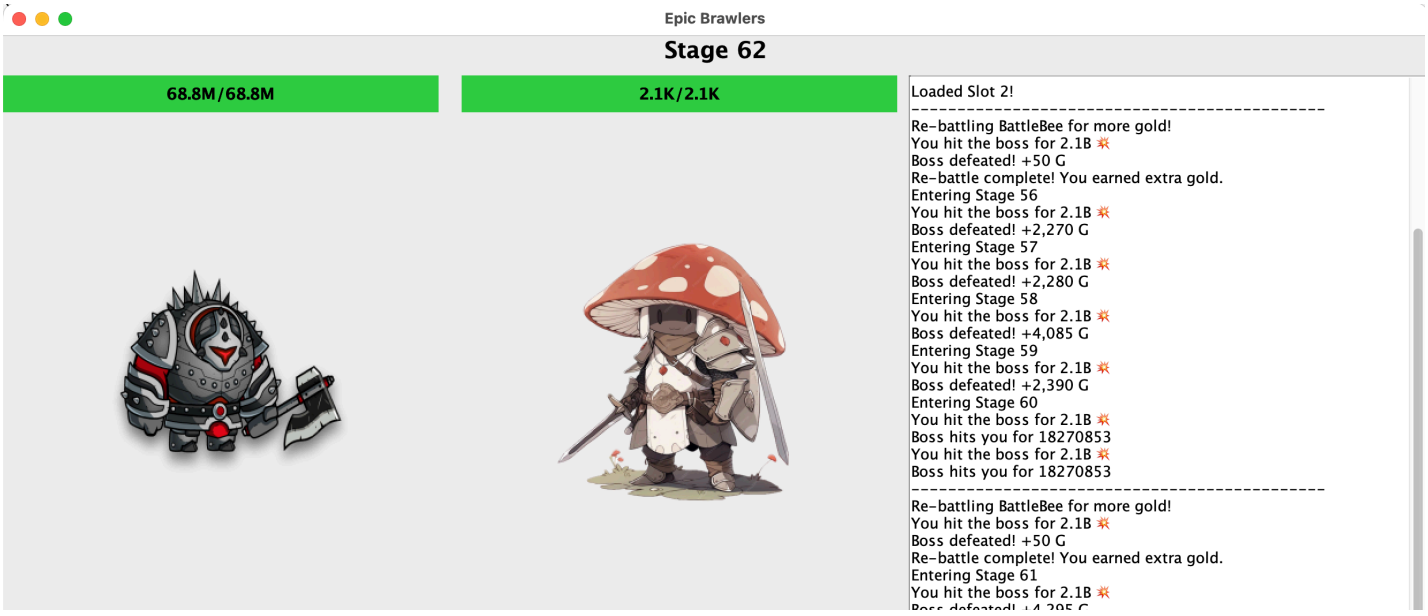
2	Implement Player and Boss classes; test damage logic and gold system.
3	Finalize model logic; improve GUI layout; possibly add intro screens to give backstory to the game; implement rough codes for all other functionalities
4	(heavy focus on GUI) Test and refine attack flow; heavy testing on GUI + include characters as well as enemies; include re-battle logic for gold accumulation; fix any logical issues with rebattling bosses
5	(focus on frontend) test the GUI for bugs + improve GUI interactivity + include better visuals; fix up shop and functionalities of shop; include option to change weapons for more damage; look at looping bosses logic
6	Fix weapon/boss overflow and scaling issues, debug the Next Stage button logic, polish UI number formatting, rebalance max HP upgrades, and refine the save system (autosave, overwrite confirmation, better slot labels).
7	lots of time debugging + improving gameplay logic before final touches.
8	Finalize all documentation and code; thorough testing

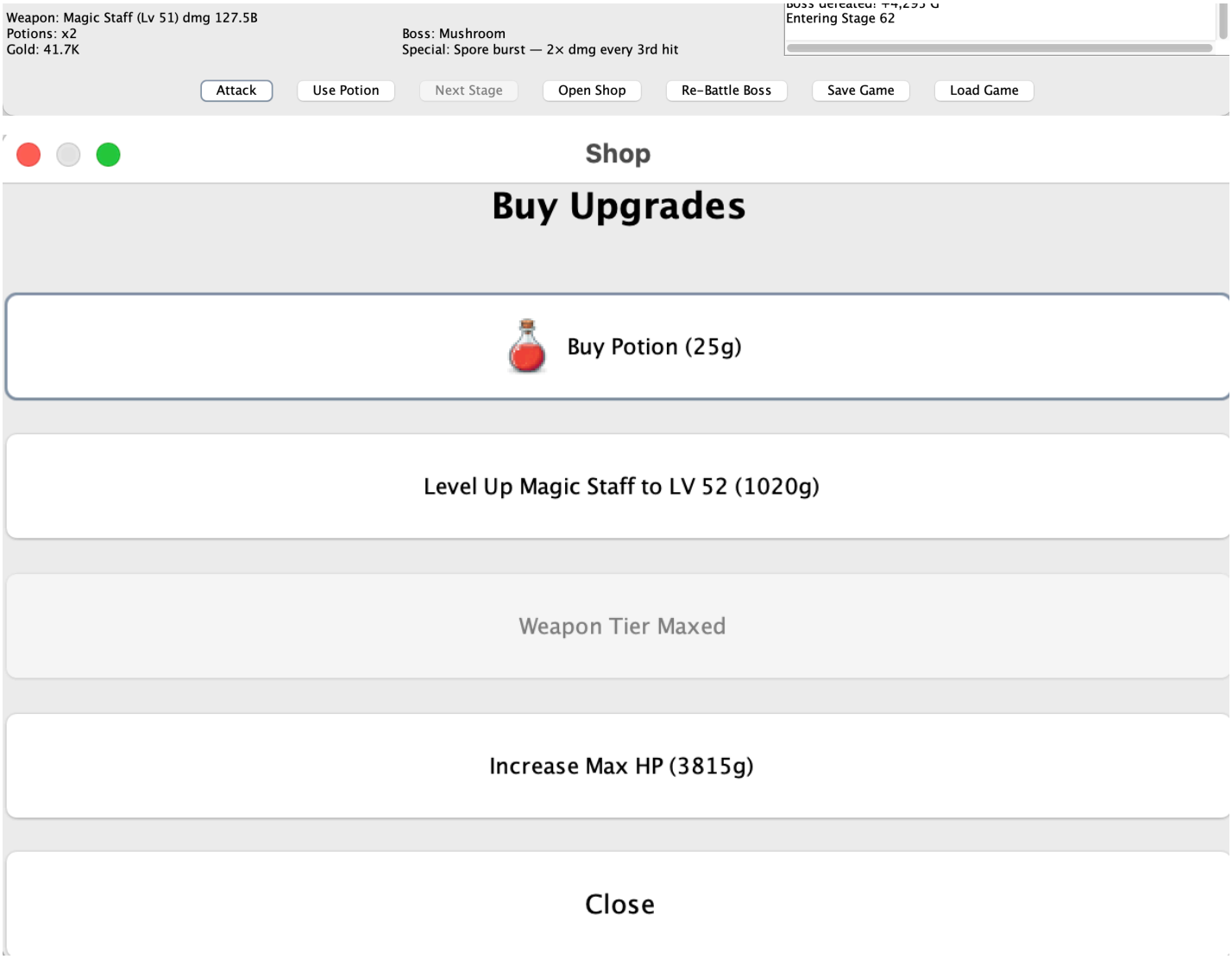
GitHub Repository

https://github.com/anhpls/epicbrawlers

Week 6: Updates

Week 6 Deliverables





Fix Log:

✓ 1. Fix Weapon Damage Overflow (Critical + Priority)

- Converted all weapon damage values from int —> long.
- Updated damage formulas to avoid overflow from exponential growth.
- Updated bosses and UI to accept long-damage without truncation.

- Fixed Magic Staff level overflow bug where damage reset to 1.

✓ 2. Fix Boss HP Scaling

- Converted boss HP from int —> long to prevent overflow at high stages.
- Updated all boss subclasses to use long HP formulas.
- Adjusted HP scaling formulas to remain realistic but challenging.
- Fixed UI HP bars to support long values without crashing.

✓ **3. Fix Next Stage Button Bug**

- Ensured enableNextStage() always triggers after boss defeat.
- Reset rebattleMode properly when entering a new stage.
- Fixed state mismatches where the button remained disabled.
- Cleaned up UI refresh ordering to stop it from getting stuck.

✓ **4. Improve UI Stats Formatting**

- Added Utils.fmt(long) to abbreviate numbers (e.g., 1.2K, 4.5M).
- Updated all stat labels to use formatted values (Gold, DMG, HP, Reward).
- Cleaned up stat layout spacing to prevent clipping on large numbers.

✓ **5. Rebalance MaxHP Upgrades**

- Added scaling cost based on number of HP upgrades purchased.
- Added percent increase scaling based on number of upgrades (no cap).
- Integrated HP upgrade count into save/load system.
- Removed hard 30% cap and ensured late-game bosses remain beatable.

▢ **6. Add Additional Save Slot Safety**

- Auto-save after every boss defeat
- Ask for confirmation before overwriting slots
- Display last played timestamp for each save slot
- Reorder the button options in popup menus (Save/Load Menus)

✓ **Prevent Potion Use at Full HP**

- Added logic so potion does **not** activate when HP is full.
- Added UI log feedback:
    - “HP already full — potion not used.”
    - “No potions left!”

✓ **Fixed Boss Damage Handling Using Long**

- Converted takeDamage(int) → takeDamage(long)
- Adjusted all subclasses to use long damage consistently.
- Fixed rounding issues caused by casting long → int.

✓ **Fixed UI Progress Bars for Long HP Values**

- Prevented bars from breaking when HP exceeded 2 billion.

✓ **Improved Save/Load**

- Added hpUpgradeCount, defeated bosses list, and long HP to SaveData.
- Fixed NullPointer issues when loading older saves.
- Rebuilt weapon reconstruction logic to prevent incorrect values.

✓ **Added Defeated Boss Tracking**

- Implemented defeatedBosses = new HashSet<>().
- Included this in save.data so that players can reload list of already defeated bosses

---

Journal Entry

This week, I focused heavily on repairing the late-game scaling system, which had caused the game to break after Stage 40–50. Many of these issues came from integer overflow, boss HP formulas collapsing, and UI elements failing to update with large values. I also completed major improvements to the shop, combat flow, logging, and UI number formatting, allowing the game to feel much more polished and stable.

Much of my time was spent reworking core systems (boss HP, weapon damage, potions, rebattle logic, UI formatting) so that the entire game could support extremely large values without breaking. I also tested and fixed several bugs involving potion usage, stage progression, boss rebattles, and save/load inconsistencies. The result is a far more playable and scalable version of the game that can now handle very high stages.

Design Changes Made This Week

1. Switched All Combat-Related Numbers to Longs (Fixing Overflow)

- Converted player damage, boss HP, rewards, and scaling formulas from int —> long.
- This prevents overflow and fixes the issue where damage or HP dropped to 1 at high stages.
- Updated bosses and weapons to use long values consistently.

2. Boss Scaling Rework (Fixing Dragon HP=1 Bug)

- Dragon HP formula now uses long scaling correctly.
- Boss HP bars updated to accept large values safely.

3. Weapon Damage System Repaired + Stabilized

- Magic Staff and Sword scaling no longer collapses at high levels.

- Damage is now stable, predictable, and properly displayed in the UI.
- Added formatting so huge damage numbers show as 1.4M, 820K, etc.

4. UI Stat Formatting Improvements

- Added new Utils.fmt(long) formatter.
- Applied it to:
    - gold
    - weapon damage
    - boss HP
    - player HP
    - log messages
- Prevents label overflow and makes stats readable at all stages.

5. Max HP Upgrade System Rebalanced

- Added hpUpgradeCount to scale cost exponentially.
- No longer a fixed 40g upgrade.
- Max HP increase % now grows as upgrades increase.
- HP upgrades feel meaningful and scale with boss difficulty.

6. Potion System Improvements

- Players can no longer waste a potion at full HP.
- New log messages:
    - “HP already full — potion not used.”
    - “No potions left!”
- Hard-to-debug potion edge cases are now patched.

7. Rebattle & Stage Progression Bug Fixes

- Fixed issue where “Next Stage” button got stuck.
- Rebattle mode now resets correctly.
- Boss defeat logic now always enables stage progression.
- Tested rebattles at high stages, now consistent.

8. Save/Load System Stability Fixes

- Added hpUpgradeCount to SaveData.
- Ensured boss HP and player long values load correctly.

- Restored weapon type/level reliably.

Challenges Encountered This Week

- Large number overflow caused damage, HP, and gold to collapse
- Dragon and other bosses scaling improperly, leading to 1 HP / unkillable states.
- UI formatting issues with large int numbers breaking labels or overflowing the GUI.
- Next Stage / rebattle logic bugs where buttons stayed disabled or didn't reset correctly.
- Potion logic edge cases, such as wasting potions at full HP or potions not updating UI.
- Save/Load incompatibilities after adding new stats (hpUpgradeCount, long HP values).
- Testing high-stage progression was difficult due to exponential scaling revealing new issues.

What I Still Need To Do

- **Add Save Slot Safety Features**
    - Confirmation prompt before overwriting save files
    - Display timestamps and cleaner labels in Load Menu
    - Improve ordering of Save/Load options
- **Test/Improve UI Final Polish**
    - Apply number formatting to all GUI elements consistently
    - Adjust label spacing so long formatted numbers never clip
- **Further Rebalance Late-Game Combat**
    - Continue tuning boss defense, HP curves, and weapon scaling beyond Stage 80+
    - Add optional items (like offensive potions) for harder bosses such as Dragons
- **Additional Game Features (Planned Add-Ons)**
    - Improve shop UI
    - Add more upgrade types (defense, crit chance, etc.)
    - Add better rebattle UI flow

Updated Timeline

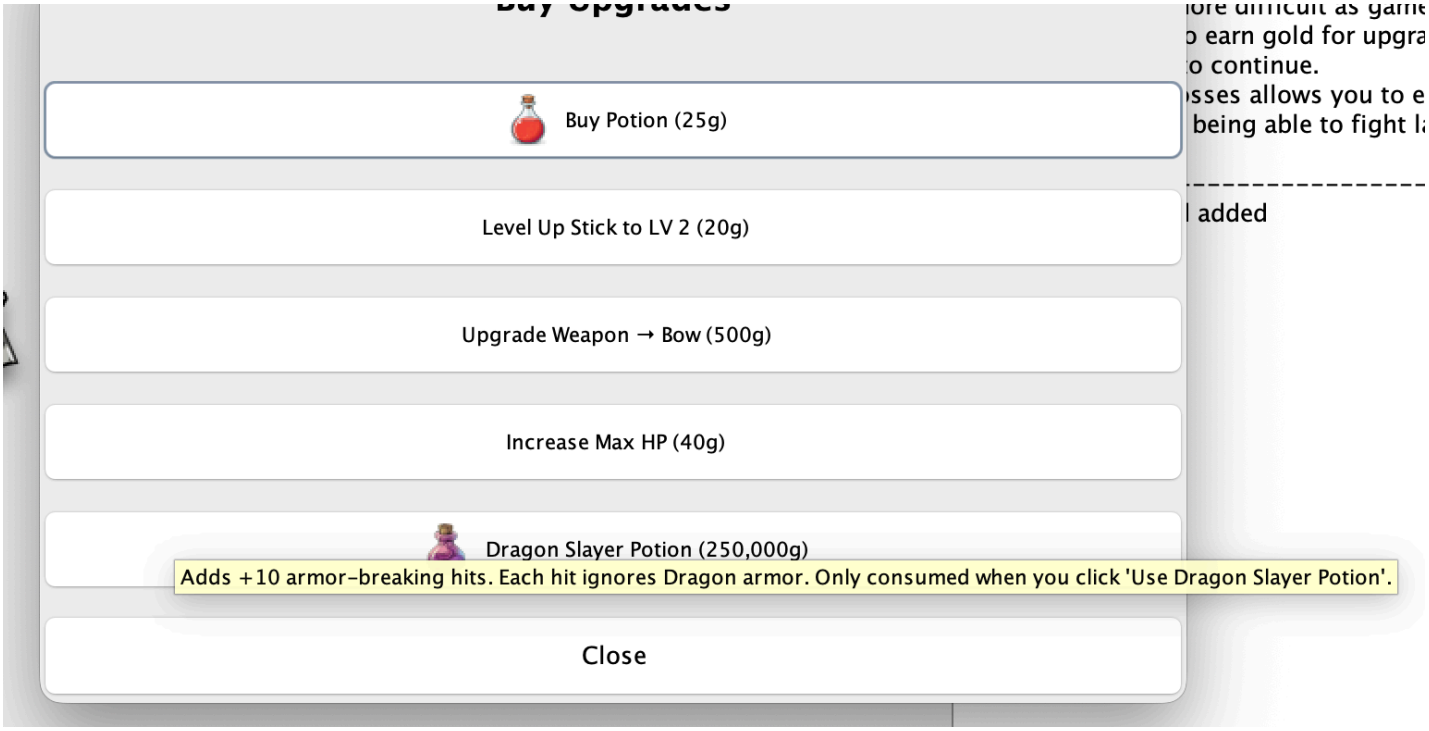
Week	Goals
1	Write proposal; design CRC + UML; set up project files
2	Implement Player/Boss classes; test attack + gold logic
3	Add more GUI elements; implement rebattling + base shop
4	GUI improvements; finalize battle flow; add early testing
5	Build Save/Load System; identify late-game errors
6	Fix overflow issues, update UI formatting, rebalance HP upgrades, repair stage progression, stabilize save/load
7	Polish, final debugging, add minor features, balance testing
8	Final documentation, final code submission

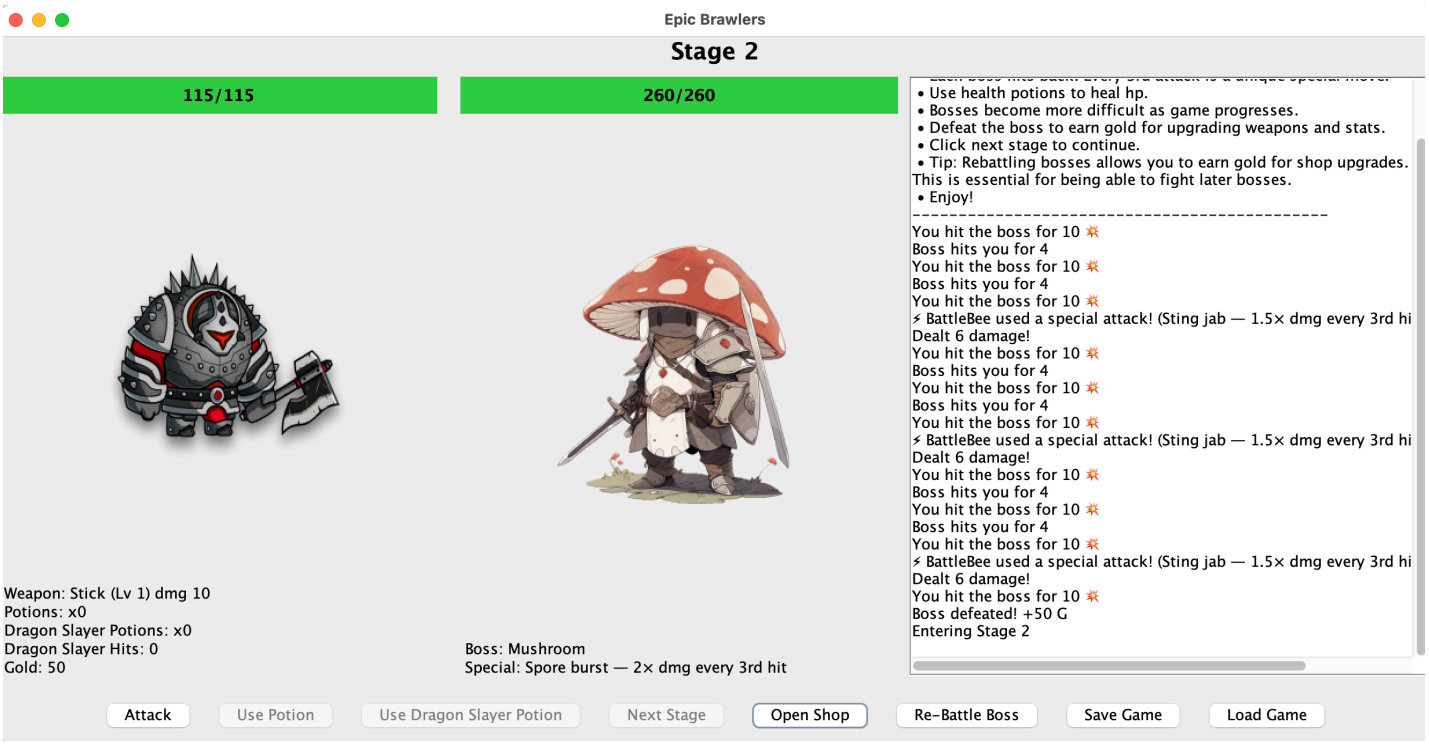
GitHub Repository

https://github.com/anhpls/epicbrawlers

Week 7: Updates

Week 7 Deliverables





Journal Entry

This week, I focused primarily on expanding the game's progression system by designing and implementing a fully functional shop and upgrade flow. Instead of only progressing by defeating bosses, players can now strategically spend gold to improve their stats and prepare for tougher encounters.

A major part of this work involved introducing a new late-game item: the Dragon Slayer Potion. I made

A major part of this week involved introducing a new late-game item, the Dragon Slayer Potion. I made sure each shop item had proper cost scaling, validation (checking if the player has enough gold), and immediate UI feedback when a purchase succeeds or fails.

I also spent time separating inventory vs. activation logic for consumables. Dragon Slayer Potions are now stored in the player's inventory and only take effect when explicitly used during battle. This required carefully updating the Player class, battle logic, and UI so potion counts, remaining hits, and button states all stay in sync.

Another focus this week was UI clarity and polish. I added stat displays for Dragon Slayer Potions and remaining armor-breaking hits, ensured buttons enable/disable correctly based on inventory, and added tooltip hover text in the shop to explain what each upgrade does. This helped reduce confusion and made the mechanics more intuitive for players.

Finally, I fixed several bugs introduced by refactoring health values to use long instead of int, including save/load issues and incorrect constructor parameters. Debugging these issues reinforced the importance of keeping data models consistent across game logic, UI, and persistence.

Overall, this week pushed the project towards a more strategic RPG experience where players must plan upgrades, manage resources, and prepare for scaling difficulty.

Design Changes Made This Week

- Introduced the Dragon Slayer Potion as a late-game mechanic to counter Dragon armor.

- Added UI stat displays for:
    - Dragon Slayer Potion count
    - Remaining Dragon Slayer hits
- Implemented tooltip hover text in the shop to explain what each upgrade does.
- Adjusted scaling and costs to better support late-game progression.
- Adjusted health and damage scaling after fixing a bug that caused instant player death at Stage 44, ensuring late-game encounters remain challenging but fair.

Challenges Encountered This Week

- Debugging an issue where the player would instantly die upon entering Stage 44, which required tracing through damage calculations and identifying overflow and scaling problems at higher stages.
- Refactoring HP values from int to long caused save/load constructor mismatches and runtime errors that required updating multiple classes to stay consistent.

- Separating potion purchase logic from activation logic introduced state synchronization issues between the Player class, BattlePanel, Shop, and ControlPanel.
- Debugging edge cases where Dragon Slayer hits were not decrementing correctly unless the Dragon boss was active.
- Ensuring UI elements stayed synchronized after purchases, stage changes, and rebattles required careful refresh ordering.

What I Still Need To Do

- Continue balancing late-game scaling now that high-stage bugs have been fixed.
- Improve visual polish of the shop and battle UI.
- Add clearer visual feedback when special mechanics (such as armor breaking) activate.
- Finalize testing of save/load functionality across multiple stages and upgrades.
- Thorough testing

Updated Timeline

Week	Goals
1	Write proposal; design CRC + UML; set up project files
2	Implement Player and Boss classes; test attack, damage, and gold logic
3	Add GUI elements; implement rebattling system; introduce basic shop framework
4	Improve GUI layout; finalize battle flow; conduct early playtesting
5	Build Save/Load system; identify late-game issues and scaling problems
6	Fix overflow issues, update UI formatting, rebalance HP upgrades, repair stage progression, stabilize save/load

7	UI polish, balance tuning for late-game bosses, additional testing and bug fixes
8	Final documentation, final testing pass, and code submission

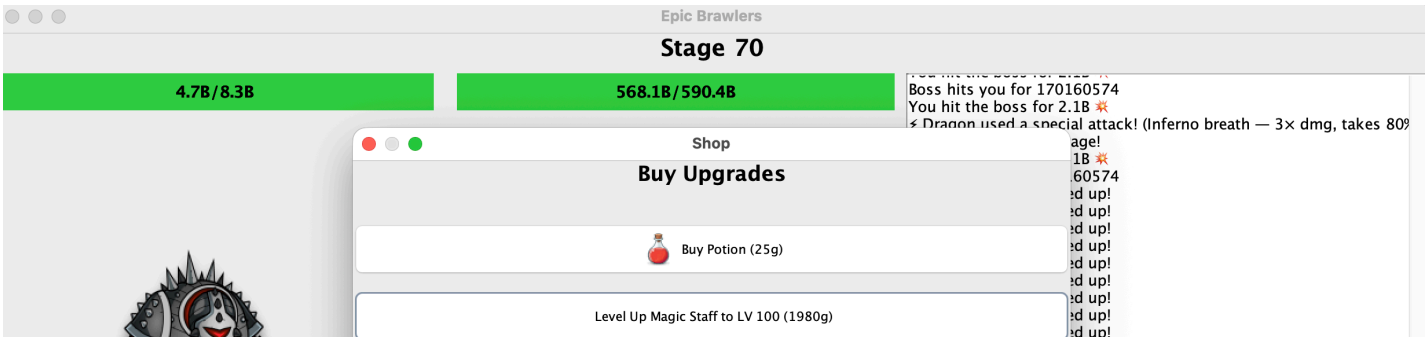
GitHub Repository

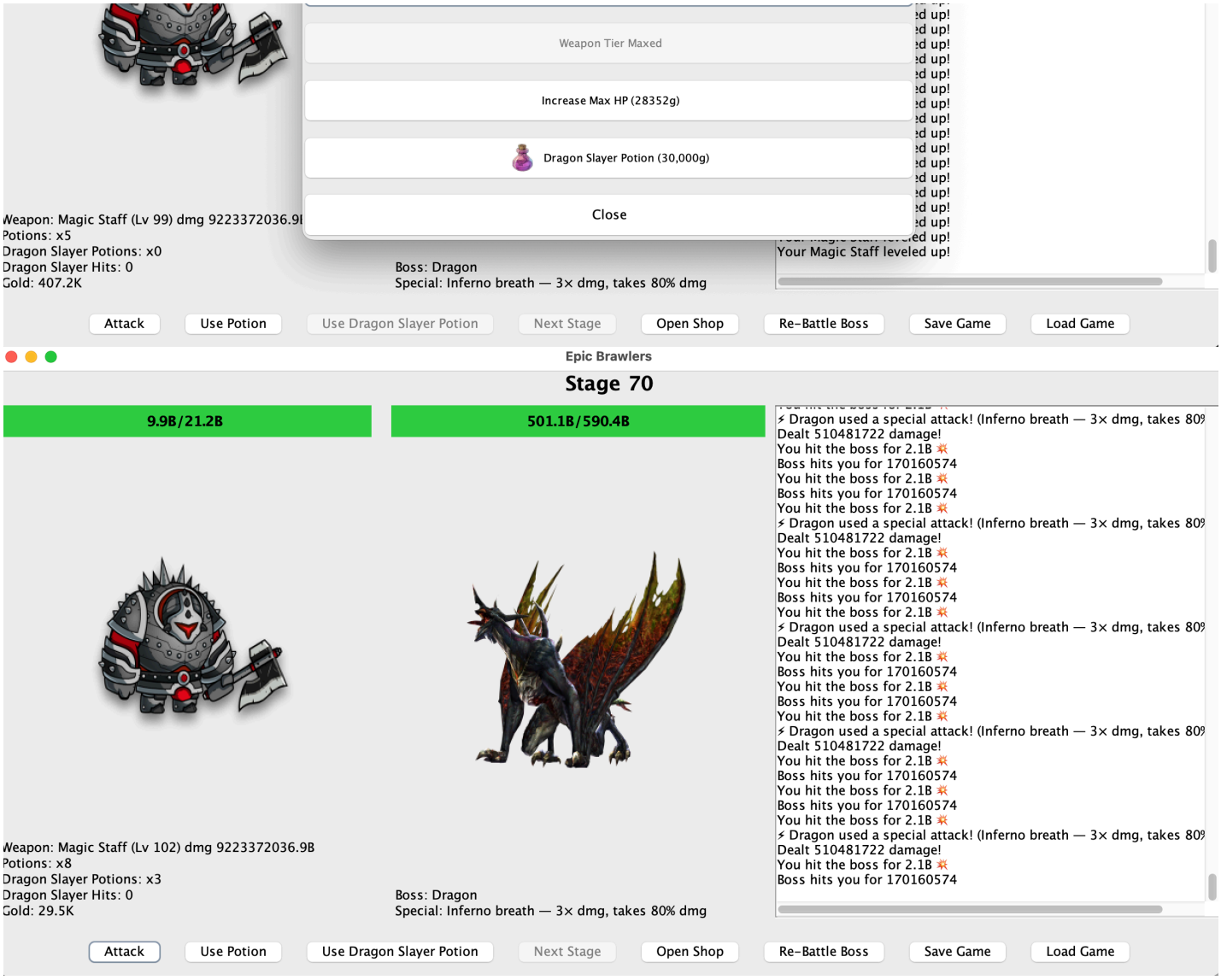
https://github.com/anhpls/epicbrawlers

Week 8: Project Wrap-up

Finish writing any other necessary code and debug (although hopefully you were debugging as you wrote the code!).

Week 8 Deliverables





Journal Entry

This week focused on final polish, documentation, and verification rather than adding new features. Since I debugged continuously as I wrote code throughout the project, there were no major issues left to resolve at this stage. Most of the core gameplay and GUI functionality was already complete, allowing me to focus on ensuring the project was clean, understandable, and well-documented.

A major task this week was finishing javadocs for each class and method, which helped clarify method responsibilities, parameters, and return values. Writing documentation also allowed me to review the code one final time and confirm that each method had a clear purpose and followed good object-oriented design principles.

I also revisited testing that I had performed more thoroughly during the previous week. Last week's testing covered gameplay flow, edge cases, and late-game behavior, including combat scaling, upgrades,

potions, stage progression, and game reset conditions. As a result, this week required minimal debugging beyond small cleanup and verification checks.

Overall, the project reached a stable and complete state, and Week 8 served as a confirmation that the design, implementation, and testing efforts from earlier weeks were successful.

Design Changes Made

No major design changes were made during this final week. The overall object-oriented structure remained consistent with earlier planning, including the separation between game logic and the GUI. Any small adjustments made were limited to minor refactoring for readability or documentation clarity.

Challenges Encountered

There were no significant new challenges during this week. The most time-consuming part was carefully reviewing the entire codebase to ensure documentation was accurate and that all functionality behaved as expected. Because debugging was handled incrementally each week, there were no last-minute issues that required major fixes.

Future Improvements

If expanded further, the project could include additional content such as new boss types, more weapon variations, enhanced GUI feedback, or expanded persistence features. These improvements would build on the existing structure without requiring major architectural changes.

What I Would Do Differently

If starting the project again, I would keep the same overall approach of incremental development and continuous debugging. This workflow prevented large issues from accumulating and made the final week significantly smoother. I would also continue prioritizing documentation earlier in the process, as it made final review and testing more efficient.

Learning Outcomes

LO	Demonstrated in Epic Brawlers
LO1: Object Oriented Design Principles	Clear separation of classes; encapsulated Player/Boss classes; GUI/controller in GameUI class
LO2: · LO2: Construct programs utilizing single and multidimensional arrays (optional)	Optional
LO3: Objects & Aggregation	GameUI aggregates Player and Boss and coordinates their interactions
LO4: Inheritance & polymorphism	Planned boss hierarchy (e.g., BaseBoss —> SlimeBoss, DragonBoss) without changing controller code
LO6: GUI & Event-Driven Programming	Swing (JFrame, JButton, JLabel, JProgressBar) with ActionListener callbacks for attacks/upgrades
LO7: Exception handling	Handling for insufficient funds on upgrade; safe HP bounds; death/reset flow
LO8: Text File I/O	Save/load player progress (gold, level) to a simple text file

GitHub Repository

https://github.com/anhpls/epicbrawlers

Video

https://youtu.be/AqKVKeuT0EQ